

GenAI & Agentic AI

Interview Preparation Guide

Questions	Topics Covered	Level	Prepared for
65+	GenAI · LangChain · LangGraph · Project	Beginner → Advanced	Infosys & MNCs

Generated: June 10, 2026 | Vishal Lazrus

Table of Contents

Section 1	Generative AI — Fundamentals	Q1–Q10
Section 2	Large Language Models (LLMs)	Q11–Q20
Section 3	LangChain — Core Concepts	Q21–Q30
Section 4	LangGraph — Architecture & Agents	Q31–Q42
Section 5	Agentic AI & Multi-Agent Systems	Q43–Q52
Section 6	ResearchForge Project — Deep Dive	Q53–Q62
Section 7	Quick Revision Summary Table	All Topics

Section 1 — Generative AI — Fundamentals

Q1 What is Generative AI? How is it different from traditional AI?

Answer:

Generative AI refers to AI models that can **create new content** — text, images, code, audio — by learning patterns from large datasets. Traditional AI is **discriminative**: it classifies or predicts from existing data. Generative AI is **generative**: it produces new data that resembles training data.

Example: A traditional spam classifier says 'this email is spam.' A generative model writes a brand new email from scratch.

Key distinction: Traditional AI = classify/predict. Generative AI = create new content.

Key Points to Remember:

- ★ Generative = creates new content (text, image, code)
- ★ Traditional AI = discriminative (classify/predict)
- ★ Learns patterns → generates similar new data
- ★ Examples: GPT-4, Gemini, DALL-E, Stable Diffusion

Q2 What is a Foundation Model?

Answer:

A Foundation Model is a **large-scale AI model pre-trained on vast amounts of data** that can be adapted (fine-tuned) for many downstream tasks. The word 'foundation' means it serves as the base for many applications.

Key properties: trained on billions of tokens, general-purpose, can be fine-tuned cheaply for specific tasks using far less data.

Examples: GPT-4, Gemini, Claude, LLaMA, Mistral — all foundation models.

Note: Fine-tuning a foundation model costs 100x less than training from scratch.

Key Points to Remember:

- ★ Pre-trained on massive data → general purpose
- ★ Can be fine-tuned for specific tasks cheaply
- ★ Examples: GPT-4, Gemini, Claude, LLaMA
- ★ Stanford coined the term in 2021

Q3**Explain the Transformer architecture in simple terms.****Answer:**

The Transformer is the architecture behind all modern LLMs. It was introduced in the 2017 paper 'Attention Is All You Need.'

Key components:

- **Tokenizer** — splits input text into tokens (words or subwords)
- **Embedding layer** — converts tokens to numerical vectors
- **Self-Attention** — lets each token 'look at' every other token to understand context
- **Feed-Forward layers** — process each token's representation
- **Output layer** — predicts the next token probability

Self-attention is what makes Transformers powerful — it captures long-range dependencies in text.

Key Points to Remember:

- ★ 2017 paper 'Attention Is All You Need' by Google
- ★ Self-attention = each word looks at all other words
- ★ Encoder-Decoder or Decoder-only architecture
- ★ BERT = Encoder only, GPT = Decoder only

Q4**What is a token? How does tokenization work?****Answer:**

A token is the basic unit of text that an LLM processes. Tokens are not always whole words — they can be word pieces, characters, or punctuation.

Example: The sentence 'Generative AI is amazing!' might be split into tokens: ['Generat','ive',' AI',' is',' amaz','ing','!'] — 7 tokens, not 5 words.

LLMs have a **context window** — the maximum number of tokens they can process at once. GPT-4 supports 128K tokens; Gemini 1.5 Pro supports 1 million tokens.

Rule of thumb: 1 token ≈ 0.75 words in English. 100 tokens ≈ 75 words.

Key Points to Remember:

- ★ Token = basic unit (not always a full word)
- ★ Tokenizer converts text → token IDs (integers)
- ★ Context window = max tokens the model can handle
- ★ More tokens = longer text but higher cost and latency

Q5

What is temperature in LLMs? What values would you use for different tasks?

Answer:

Temperature controls the **randomness of the model's output**. It scales the probability distribution of the next token before sampling.

Low temperature (0.0–0.3): output is deterministic and focused. The model picks the most likely token every time.

High temperature (0.7–1.0+): output is more random and creative. The model sometimes picks less likely tokens.

Temperature 0 = always picks the most probable token = deterministic output.

Use cases:

- temperature=0 → code generation, structured data extraction, quality scoring (need consistency)
- temperature=0.3–0.5 → analysis, summarisation
- temperature=0.7–1.0 → creative writing, brainstorming

Key Points to Remember:

- ★ Temperature = controls randomness of output
- ★ 0 = deterministic, 1+ = very random/creative
- ★ Use 0 for code/facts, 0.7+ for creative tasks
- ★ In ResearchForge: planner uses 0.3, quality gate uses 0

Q6

What is Prompt Engineering? Name key techniques.

Answer:

Prompt Engineering is the practice of designing and optimizing the input text (prompt) sent to an LLM to reliably get the desired output format, tone, and content.

Key techniques:

- **Zero-shot prompting** — just ask the question with no examples
- **Few-shot prompting** — give 2–3 examples before the actual question
- **Chain-of-Thought (CoT)** — ask the model to 'think step by step'
- **Role prompting** — 'You are an expert researcher...'
- **Structured output** — specify exact format: 'Reply as: TASK N | agent | description'

In ResearchForge: planner.py uses structured output prompting to get 'TASK 1 | search_agent | ...' format for reliable parsing.

Key Points to Remember:

- ★ Zero-shot, Few-shot, Chain-of-Thought, Role prompting
- ★ Structured output = specify exact format in prompt
- ★ Better prompts = more consistent, parseable responses
- ★ Prompt is the primary way to control LLM behaviour

Q7

What is RAG (Retrieval-Augmented Generation)?

Answer:

RAG is a technique that combines a **retrieval system with an LLM**. Instead of relying only on the LLM's training data, RAG first retrieves relevant documents from an external knowledge base, then passes them to the LLM as context.

Flow: User query → search vector database → retrieve top-K relevant documents → pass documents + query to LLM → LLM generates answer grounded in the retrieved data.

RAG solves two LLM problems: hallucination (making up facts) and knowledge cutoff (outdated training data).

Tools used: FAISS, Pinecone, ChromaDB (vector stores) + Tavily/Google (web retrieval).

Note: ResearchForge uses web search (Tavily) as a live retrieval step — similar to RAG but with real-time web data instead of a static vector database.

Key Points to Remember:

- ★ RAG = Retrieval + LLM generation
- ★ Solves hallucination and knowledge cutoff
- ★ Vector DB stores document embeddings for retrieval
- ★ More accurate answers than pure LLM

Q8

What is Hallucination in LLMs? How do you reduce it?

Answer:

Hallucination is when an LLM generates text that is **factually incorrect but sounds confident and plausible**. The model 'makes up' information not supported by its training data.

Why it happens: LLMs learn to predict the next token statistically, not to retrieve verified facts. They optimise for fluency, not accuracy.

How to reduce it:

- Use RAG — ground answers in retrieved documents
- Lower temperature — reduces random token selection
- Ask model to cite sources or say 'I don't know'
- Use LLM-as-Judge to verify output (like ResearchForge's quality gate)

In ResearchForge: the quality gate acts as a hallucination check by scoring the report and retrying if quality is low.

Key Points to Remember:

- ★ Hallucination = confident but wrong output
- ★ Cause: statistical token prediction, not fact retrieval
- ★ Fix: RAG, lower temperature, LLM-as-Judge, citations
- ★ Hallucination rate varies by model and task

Q9**What is Fine-tuning? When would you use it vs prompting?****Answer:**

Fine-tuning is the process of **taking a pre-trained foundation model and training it further on a small, task-specific dataset** to make it better at that specific task.

Analogy: hiring an expert (foundation model) and then giving them 2 weeks of company-specific training (fine-tuning).

Use fine-tuning when: you need consistent domain-specific style or knowledge, prompting alone cannot achieve accuracy, you have labelled training data.

Use prompting when: task changes frequently, data is limited, speed of iteration matters, good enough accuracy is achievable.

In most production use cases today: prompt engineering + RAG is tried first. Fine-tuning is done only when those are insufficient.

Key Points to Remember:

- ★ Fine-tuning = extra training on task-specific data
- ★ Use when: consistent style, domain knowledge, have labelled data
- ★ Use prompting when: fast iteration, limited data
- ★ Fine-tuning is expensive; try prompting + RAG first

Q10**What is the difference between Generative AI and Predictive AI?****Answer:**

Predictive AI learns patterns to forecast or classify: stock prices, spam detection, disease diagnosis. It maps inputs to predefined output categories or values.

Generative AI learns the underlying distribution of data to create new instances: write a poem, generate an image, produce code.

Predictive AI = 'what will happen?' | Generative AI = 'create something new'

Both use neural networks but with different objectives. Predictive uses classification/regression loss. Generative uses likelihood maximisation or diffusion objectives.

Key Points to Remember:

- ★ Predictive = classify or forecast from existing categories
- ★ Generative = create new content from learned distribution
- ★ Both use neural networks, different training objectives
- ★ Modern LLMs blur the line (can both generate and classify)

Section 2 — Large Language Models (LLMs)

Q11 What is an LLM? How does it generate text?

Answer:

An LLM (Large Language Model) is a neural network with billions of parameters trained on massive text datasets to understand and generate human language.

Text generation process: The model takes input tokens, processes them through transformer layers with self-attention, and at each step predicts a probability distribution over its entire vocabulary. It samples the next token from this distribution, adds it to the input, and repeats until a stop condition is met. This is called **autoregressive generation**.

Key: LLMs predict ONE token at a time, not the whole sentence at once.

Key Points to Remember:

- ★ Billions of parameters trained on text
- ★ Autoregressive = generates one token at a time
- ★ Self-attention captures relationships between tokens
- ★ Vocabulary size typically 30K–100K tokens

Q12 What is the difference between GPT, BERT, and T5?

Answer:

These are different Transformer architectures with different training objectives:

- **BERT** (Encoder-only) — reads the full sentence bidirectionally. Best for: understanding, classification, Q&A, embeddings. Cannot generate text.
- **GPT** (Decoder-only) — reads left to right, generates text autoregressively. Best for: text generation, chat, code. Used by ChatGPT.
- **T5** (Encoder-Decoder) — converts every task to text-to-text format. Best for: translation, summarisation, question answering.

For interview: BERT = understand, GPT = generate, T5 = transform one text to another.

Key Points to Remember:

- ★ BERT = encoder only = good at understanding/classification
- ★ GPT = decoder only = good at generation
- ★ T5 = encoder-decoder = text-to-text tasks
- ★ Modern LLMs are mostly decoder-only (GPT architecture)

Q13**What is the context window and why does it matter?****Answer:**

The context window is the maximum number of tokens an LLM can process in a single request — both input and output combined.

Why it matters: The LLM can only 'see' and reason about text within its context window. Information outside it is invisible to the model.

Context window sizes: GPT-3.5 = 4K, GPT-4 = 128K, Gemini 1.5 Pro = 1M, Claude 3.5 Sonnet = 200K tokens.

ResearchForge solves context window limits with a Virtual File System — agents store intermediate results externally instead of keeping everything in the prompt.

Key Points to Remember:

- ★ Max tokens model can see at once
- ★ Larger context = more expensive and slower
- ★ Virtual File System = workaround for context limits
- ★ 1 page ≈ 500 tokens, 1 novel ≈ 100K tokens

Q14**What is Embedding? How are embeddings used in AI?****Answer:**

An embedding is a **fixed-size numerical vector that represents the meaning of text**. Similar texts have similar (close) vectors in the embedding space.

Example: 'king' - 'man' + 'woman' ≈ 'queen' in embedding space — the arithmetic works because embeddings encode semantic meaning.

Uses: semantic search, RAG (find similar documents), clustering, classification, recommendation systems.

How it works: pass text through an embedding model (e.g., text-embedding-ada-002) → get a vector of 1536 floats → store in a vector database → compare with cosine similarity.

Embedding = convert text to numbers that capture meaning. Similar meaning → similar numbers.

Key Points to Remember:

- ★ Embedding = numerical vector representing meaning
- ★ Similar text = similar vectors (close in space)
- ★ Used in RAG, semantic search, clustering
- ★ Cosine similarity measures how similar two embeddings are

Q15

What is the difference between zero-shot, one-shot, and few-shot learning?

Answer:

These terms describe how many examples are provided to the model in the prompt:

- **Zero-shot** — no examples given. Just the instruction. Relies entirely on the model's pre-training.
'Translate this to French: Hello'
- **One-shot** — one example given before the actual task. Shows the model what format is expected.
- **Few-shot** — 2–10 examples given. More examples = better performance on novel tasks.

Few-shot prompting is powerful because it adapts the model's behaviour without any training — just examples in the prompt.

Note: In ResearchForge, the planner prompt uses structured output format — a form of zero-shot structured prompting.

Key Points to Remember:

- ★ Zero-shot = no examples, relies on pre-training
- ★ Few-shot = 2-10 examples in prompt
- ★ More examples = better format adherence
- ★ No training needed — just prompt engineering

Q16

What is RLHF (Reinforcement Learning from Human Feedback)?

Answer:

RLHF is the technique used to align LLMs with human preferences. It involves three steps:

- **Step 1 — Supervised Fine-Tuning (SFT):** Train on high-quality human-written demonstrations
- **Step 2 — Reward Model Training:** Humans rank multiple model outputs → train a 'reward model' that scores how good a response is
- **Step 3 — PPO (Reinforcement Learning):** Use the reward model as a feedback signal to update the LLM to generate higher-scoring responses

RLHF is why ChatGPT feels helpful and safe — it was trained to align with human preferences, not just predict text.

Used by: OpenAI (ChatGPT), Anthropic (Claude), Google (Gemini)

Key Points to Remember:

- ★ RLHF = trains LLM to align with human preferences
- ★ Three steps: SFT → Reward Model → PPO
- ★ Makes models helpful, harmless, honest
- ★ Without RLHF: raw LLMs can be harmful/unhelpful

Q17**What is the difference between GPT-3.5 and GPT-4?****Answer:**

GPT-4 is significantly more capable than GPT-3.5 across multiple dimensions:

- **Size:** GPT-4 is rumoured to be a Mixture-of-Experts model (~1.8T parameters). GPT-3.5 has 175B parameters.
- **Reasoning:** GPT-4 scores much higher on standardised tests (BAR exam, SAT, etc.)
- **Context window:** GPT-4 Turbo = 128K tokens vs GPT-3.5 = 4–16K
- **Multimodal:** GPT-4 can process images as input. GPT-3.5 is text-only.
- **Cost:** GPT-4 is 10-30x more expensive per token than GPT-3.5

For interviews: always mention the capability vs cost tradeoff — use GPT-3.5/Flash for simple tasks, GPT-4 for complex reasoning.

Key Points to Remember:

- ★ GPT-4 = better reasoning, larger context, multimodal
- ★ GPT-3.5 = cheaper, faster, good enough for many tasks
- ★ Tradeoff: capability vs cost vs latency
- ★ Gemini 2.5 Flash = Google's GPT-3.5 equivalent (free tier)

Q18**What is a System Prompt? How is it different from a User Prompt?****Answer:**

In chat-based LLMs, there are typically three message roles:

- **System prompt** — instructions set by the developer/operator. Defines the AI's persona, constraints, and task. The user typically cannot see or override it.
- **User prompt** — the message from the end user in the conversation.
- **Assistant message** — the model's response.

System prompt = the 'briefing' you give the AI before the conversation starts. User prompt = what the user actually types.

In ResearchForge: each agent node builds a detailed system-like prompt that tells Gemini its role ('You are a research analysis expert') and exact output format.

Key Points to Remember:

- ★ System prompt = developer instructions, sets AI behaviour
- ★ User prompt = what the end user types
- ★ System prompt cannot be overridden by user
- ★ Good system prompts = consistent, reliable output

Q19**What is the difference between open-source and closed-source LLMs?****Answer:**

Closed-source LLMs: weights are not publicly available, accessed only via API, company controls access and pricing.

Examples: GPT-4 (OpenAI), Gemini (Google), Claude (Anthropic)

Open-source LLMs: weights are publicly available, can be downloaded and run locally, can be fine-tuned freely.

Examples: LLaMA 3 (Meta), Mistral, Falcon, Phi-3

For production: open-source models via Ollama = free, private, no rate limits. Closed-source APIs = easier to use, better performance, costs money.

Note: ResearchForge supports both — Gemini API (closed) and Ollama with LLaMA (open-source local).

Key Points to Remember:

- ★ Closed = API only, company controls access
- ★ Open = weights public, can run locally
- ★ Open-source = free, private, customisable
- ★ LLaMA 3, Mistral = popular open-source LLMs

Q20**What are the main challenges in deploying LLMs in production?****Answer:**

Deploying LLMs at production scale involves several engineering challenges:

- **Latency:** LLMs generate tokens slowly. Solutions: streaming, smaller/quantised models, edge deployment
- **Cost:** API costs scale with token count. Solutions: caching, smaller models, batching
- **Hallucination:** LLMs make up facts. Solutions: RAG, citations, LLM-as-Judge
- **Context window limits:** Long documents exceed limits. Solutions: chunking, summarisation, virtual file systems
- **Rate limits:** Free tier APIs throttle requests. Solutions: retry logic, multiple providers, local models

In ResearchForge: rate limits from Gemini free tier led to adding Groq and Ollama as fallback providers.

Key Points to Remember:

- ★ Main challenges: latency, cost, hallucination, context limits
- ★ Rate limits = major issue on free-tier APIs
- ★ Solutions: caching, RAG, streaming, fallback providers
- ★ Always design with failure cases in mind

Section 3 — LangChain — Core Concepts

Q21 What is LangChain? Why was it created?

Answer:

LangChain is an open-source Python (and JavaScript) framework that makes it easier to build applications powered by LLMs. It provides standardised abstractions for common patterns: connecting to LLMs, managing prompts, chaining steps together, and integrating tools.

Why it was created: before LangChain, developers wrote raw API calls to OpenAI, managed prompts manually, and built retrieval pipelines from scratch. LangChain standardised these patterns so developers could build faster.

Think of LangChain as 'the Django/Spring framework for LLM applications' — it handles boilerplate so you focus on your actual application.

Key Points to Remember:

- ★ Framework for building LLM-powered applications
- ★ Standardises: LLM calls, prompts, chains, tools, memory
- ★ Created by Harrison Chase in 2022
- ★ Python + JavaScript, open-source

Q22 What is a Chain in LangChain?

Answer:

A Chain in LangChain is a **sequence of components that run in order**, where the output of one step becomes the input to the next. It is the simplest way to build multi-step LLM pipelines.

Example: PromptTemplate → LLM → OutputParser — this is a basic chain: format the prompt, send to LLM, parse the response.

LangChain Expression Language (LCEL) uses the pipe operator | to chain components:

```
chain = prompt | llm | output_parser
result = chain.invoke({'topic': 'AI'})
```

Chain = step1 | step2 | step3 — output flows left to right.

Key Points to Remember:

- ★ Chain = sequence of steps, output feeds into next
- ★ LCEL uses pipe | operator to connect components
- ★ Most basic: PromptTemplate | LLM | OutputParser
- ★ For complex branching/loops → use LangGraph

Q23

What is a Prompt Template in LangChain?

Answer:

A PromptTemplate is a reusable template for constructing prompts with variable substitution. Instead of hardcoding prompts, you define a template with placeholders and fill them at runtime.

```
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages([
    ('system', 'You are a research expert.'),
    ('human', 'Research this topic: {topic}')
])

prompt = template.invoke({'topic': 'AI in healthcare'})
```

PromptTemplates make prompts reusable, testable, and easier to version-control.

Key Points to Remember:

- ★ PromptTemplate = reusable prompt with variable slots
- ★ Separates prompt logic from business logic
- ★ ChatPromptTemplate for chat models (system/human/AI roles)
- ★ Can be chained with | to LLM directly

Q24

What are Tools in LangChain? How do agents use them?

Answer:

A Tool in LangChain is a **function that an AI agent can call** to interact with the external world. Tools give agents the ability to search the web, run code, query databases, send emails, etc.

A tool has: a name, a description (tells the LLM what it does), and a function (the actual code to run).

The LLM decides WHICH tool to call based on the tool description and the current task. This is called Tool Use or Function Calling.

Tool = function + description. The description is what the LLM reads to decide whether to call it.

In ResearchForge: `search_web()` is effectively a tool — it takes a query and returns web results.

Key Points to Remember:

- ★ Tool = named function an LLM agent can call
- ★ LLM reads tool description to decide when to call it
- ★ Examples: web search, code execution, database query
- ★ Tool description quality = critical for correct tool selection

Q25

What is Memory in LangChain?

Answer:

Memory allows an LLM application to remember information across multiple conversation turns or steps. Without memory, every LLM call is stateless — the model has no idea what was said before.

Types of memory:

- **ConversationBufferMemory** — stores the full conversation history
- **ConversationSummaryMemory** — summarises older messages to save tokens
- **VectorStoreMemory** — stores and retrieves memories by semantic similarity

In LangGraph, memory is handled by the **State** — the shared dictionary passed between nodes that persists all information across the entire pipeline.

LangGraph State = the most robust form of memory for multi-agent systems.

Key Points to Remember:

- ★ Memory = persistence across conversation turns
- ★ Without memory = every call starts fresh
- ★ LangGraph State = best memory for agent pipelines
- ★ Buffer memory (full history) vs Summary memory (compressed)

Q26

What is the difference between LangChain and LangGraph?

Answer:

LangChain is for building linear pipelines: $A \rightarrow B \rightarrow C$. It is ideal for simple chains where the flow is always the same.

LangGraph extends LangChain specifically for building **stateful, graph-based workflows** that support loops, branches, and multiple agents sharing state.

- **LangChain**: simple chains, no loops, no shared state, synchronous steps
- **LangGraph**: graph with nodes/edges, supports loops, shared TypedDict state, conditional routing

Use LangChain for simple pipelines. Use LangGraph when you need loops, retry logic, or multiple agents collaborating.

Note: ResearchForge uses LangGraph because the quality gate needs to loop back to the synthesizer — impossible with plain LangChain.

Key Points to Remember:

- ★ LangChain = linear chain $A \rightarrow B \rightarrow C$
- ★ LangGraph = graph with loops, branches, shared state
- ★ LangGraph built on top of LangChain
- ★ Choose LangGraph when you need loops or multi-agent

Q27 What is an Output Parser in LangChain?

Answer:

An Output Parser processes the raw string output from an LLM and converts it into a structured Python object (dict, list, Pydantic model, etc.).

Why needed: LLMs return text strings. Applications often need structured data (JSON, list of items, custom objects).

Common parsers: StrOutputParser (raw string), JsonOutputParser, PydanticOutputParser (validated schema), CommaSeparatedListOutputParser.

In ResearchForge: `parse_todos()` is a custom output parser — it splits the LLM's response by | character and builds a list of todo dictionaries.

```
# Custom parser in planner.py
for line in llm_response.split('\n'):
    parts = line.split('|')
    todos.append({'task': parts[2], 'agent': parts[1]})
```

Key Points to Remember:

- ★ Output Parser = converts LLM string output to Python objects
- ★ LLMs return text → parsers give structure
- ★ Common: JSON, Pydantic, CSV, custom
- ★ Always validate parsed output — LLMs can deviate from format

Q28 What are Callbacks in LangChain?

Answer:

Callbacks are hooks that run at specific events during LangChain execution — useful for logging, monitoring, and debugging.

Common events: `on_llm_start` (when LLM receives prompt), `on_llm_end` (when LLM returns response), `on_chain_start`, `on_tool_start`, `on_agent_action`.

LangSmith — the observability platform — uses callbacks internally to capture every LLM call, token count, latency, and output automatically.

LangSmith + callbacks = full X-ray view of your agent's execution. Essential for debugging multi-step pipelines.

Key Points to Remember:

- ★ Callbacks = hooks at LLM/chain/tool events
- ★ Used for logging, monitoring, tracing
- ★ LangSmith uses callbacks to trace every step
- ★ Set `LANGCHAIN_TRACING_V2=true` to enable tracing

Q29

What is LangSmith and why is it important?

Answer:

LangSmith is Anthropic/LangChain's observability platform for LLM applications. It automatically traces every LLM call, tool use, and chain execution — capturing inputs, outputs, token counts, latencies, and errors.

Why important for development:

- See exactly what prompt was sent to the LLM at every step
- Debug agent failures — see which step went wrong and why
- Monitor costs — see token usage across all calls
- Run evaluations — test prompt changes against datasets

For ResearchForge: LangSmith shows the full trace of all 4 nodes, what each agent was told, and what it responded. Invaluable for debugging.

Note: Enable with: `LANGCHAIN_TRACING_V2=true` and `LANGCHAIN_API_KEY` in `.env`

Key Points to Remember:

- ★ LangSmith = observability/tracing for LLM apps
- ★ Traces: inputs, outputs, tokens, latency, errors
- ★ Essential for debugging multi-agent pipelines
- ★ Free dev tier available at smith.langchain.com

Q30

What is a Vector Database? How does it work with LangChain?

Answer:

A Vector Database stores embeddings (numerical vectors representing text meaning) and enables fast similarity search — find the most similar vectors to a query vector.

How it works: (1) Convert documents to embeddings, (2) store in vector DB, (3) at query time convert query to embedding, (4) find nearest vectors using cosine or dot product similarity, (5) return the corresponding documents.

Popular options: FAISS (Facebook, local), Pinecone (cloud, managed), ChromaDB (local, open-source), Weaviate, Qdrant.

Vector DB + LangChain = the foundation of most RAG systems.

LangChain integration: `from langchain_community.vectorstores import FAISS`

Key Points to Remember:

- ★ Vector DB = stores embeddings, enables similarity search
- ★ Core of RAG systems
- ★ Popular: FAISS, Pinecone, ChromaDB
- ★ Cosine similarity = how similar two vectors are (0-1)

Section 4 — LangGraph — Architecture & Agents

Q31 What is LangGraph? What problem does it solve?

Answer:

LangGraph is a library built on top of LangChain for building **stateful, multi-step AI workflows** as graphs. It treats your AI pipeline as a directed graph where nodes are processing steps and edges define the flow.

Problems it solves:

- LangChain chains are linear — LangGraph adds loops and branches
- No shared memory across steps — LangGraph State solves this
- Hard to build multi-agent systems — LangGraph makes it structured
- No retry/self-correction logic — conditional edges enable this

LangGraph is the standard framework for production-grade agentic AI systems in 2024-2025.

Key Points to Remember:

- ★ Graph-based framework for stateful AI workflows
- ★ Adds loops, branches, shared state to LangChain
- ★ Standard for production multi-agent systems
- ★ Built by LangChain team, open-source

Q32 Explain StateGraph and how State works in LangGraph.

Answer:

StateGraph is the core class in LangGraph. You initialise it with a TypedDict schema that defines all the fields in the shared state.

How State works: every node receives the full state dictionary, can read any field, and returns a dictionary with only the fields it wants to update. LangGraph merges these updates back into the full state automatically.

```
from langgraph.graph import StateGraph
from typing import TypedDict

class MyState(TypedDict):
    query: str
    result: str

graph = StateGraph(MyState)
graph.add_node('step1', my_function)
graph.set_entry_point('step1')
```

State = the shared whiteboard. Every node reads the whole whiteboard. Returns only its changes.

Key Points to Remember:

- ★ StateGraph = LangGraph's core class
- ★ TypedDict defines all fields agents can read/write
- ★ Node reads full state, returns only updated fields
- ★ LangGraph auto-merges updates into full state

Q33**What are Nodes in LangGraph? What is the function signature?****Answer:**

A node in LangGraph is a Python function that takes the current state and returns an updated state. That is the complete definition.

```
def my_node(state: MyState) -> MyState:
    # Read from state
    query = state['query']

    # Do work (call LLM, search, etc.)
    result = llm.invoke(query)

    # Return ONLY the fields you changed
    # LangGraph merges this with existing state
    return {'result': result.content}
```

Important: you return a partial dict with only changed fields. The `{**state, 'key': value}` pattern includes all existing fields plus your changes.

Every single function in `graph/planner.py`, `workers.py`, `synthesizer.py`, `quality_gate.py` follows this exact pattern.

Key Points to Remember:

- ★ Node = Python function (state) -> state
- ★ Read anything from state, return only what you changed
- ★ LangGraph merges updates automatically
- ★ Registered with: `graph.add_node('name', function)`

Answer:

Edges define how the graph flows from one node to the next. LangGraph has two types:

- **Regular edge** — always goes from node A to node B, no conditions. Added with `graph.add_edge('A', 'B')`.
- **Conditional edge** — calls a router function after node A. The router returns a string that maps to the next node name.

```
# Regular edge — always happens
graph.add_edge('planner', 'workers')

# Conditional edge — depends on state
graph.add_conditional_edges(
    'quality_gate',    # from this node
    should_retry,      # router function
    {'retry': 'synthesizer', 'done': END}
)
```

Conditional edges are what make LangGraph powerful — they enable loops, retry logic, and dynamic routing.

Key Points to Remember:

- ★ Regular edge = always go from A to B
- ★ Conditional edge = router function decides next node
- ★ Router function returns a string key
- ★ String key maps to node name in dict

Q35

What is `set_entry_point` and `END` in `LangGraph`?**Answer:**

`set_entry_point('node_name')` tells `LangGraph` which node to run first when `graph.invoke()` is called. Every graph must have exactly one entry point.

`END` is a special constant imported from `langgraph.graph`. When a conditional edge routes to `END`, the graph stops executing and returns the final state.

```
from langgraph.graph import StateGraph, END

graph.set_entry_point('planner') # start here

# End condition
graph.add_conditional_edges(
    'quality_gate', router,
    {'done': END, 'retry': 'synthesizer'}
)
```

Without `END` in any path, the graph would run forever. Always ensure every possible path eventually reaches `END`.

Key Points to Remember:

- ★ `set_entry_point` = first node to run
- ★ `END` = special constant that terminates the graph
- ★ Every graph must have at least one path to `END`
- ★ `graph.compile()` validates the graph structure

Q36

What is `graph.compile()`? What does it do?**Answer:**

`compile()` validates the graph structure and returns a `Runnable` object that can be invoked, streamed, or batched. It checks that all nodes referenced in edges actually exist and that the graph is valid.

After compilation, the graph becomes immutable — you cannot add more nodes or edges. The compiled graph exposes these methods:

- `.invoke(state)` — run synchronously, wait for full result
- `.stream(state)` — run and yield state after each node
- `.ainvoke(state)` — async version of `invoke`
- `.astream(state)` — async streaming

```
runnable = graph.compile()
final_state = runnable.invoke(initial_state)
```

`compile()` = lock the graph and make it runnable. Always call this before `invoke()`.

Key Points to Remember:

- ★ `compile()` validates and locks the graph
- ★ Returns `Runnable` with `.invoke()` / `.stream()`
- ★ Cannot modify graph after compile
- ★ `.stream()` lets you see state after each node

Q37 How does LangGraph handle the retry/loop pattern?

Answer:

Loops in LangGraph are implemented using conditional edges that route back to an earlier node. The router function checks the current state and decides whether to loop or exit.

Important: you must track iteration count in state to avoid infinite loops. Always add a `max_retries` guard.

```
def should_retry(state):
    score = state['quality_score']
    retries = state['retry_count']

    # Guard: max 2 retries
    if score < 5.0 and retries < 2:
        return 'retry'    # loop back
    return 'done'        # exit to END
```

Loop = conditional edge that points back to an earlier node. Always add `retry_count` guard.

Note: In ResearchForge: synthesizer increments `retry_count` each time. `quality_gate` checks it. Max 2 retries prevents infinite loops.

Key Points to Remember:

- ★ Loop = conditional edge pointing back to earlier node
- ★ Always track `retry_count` in state
- ★ Always add `max_retries` guard
- ★ Without guard: infinite loop = app hangs forever

Q38 What is the difference between `graph.invoke()` and `graph.stream()`?

Answer:

`.invoke(initial_state)` runs the entire graph synchronously and returns the final state only after ALL nodes have finished. You get nothing until the graph completes.

`.stream(initial_state)` runs the graph and yields an update dictionary after EACH node completes. You can process intermediate results in real-time.

```
# invoke - wait for everything
final = graph.invoke(state)

# stream - see each step
for update in graph.stream(state):
    node_name = list(update.keys())[0]
    print(f'{node_name} completed')
```

For Streamlit: use `.invoke()` for simplicity. For production UIs that show live progress, use `.stream()` to update the UI after each node.

Key Points to Remember:

- ★ `invoke()` = run all, return only final state
- ★ `stream()` = yield update after each node
- ★ `stream()` enables real-time progress updates in UI
- ★ Both return same final state

Answer:

Checkpointing allows LangGraph to **save the graph state after every node execution** to a persistent store (SQLite, PostgreSQL, Redis). This enables:

- **Resume on failure** — if the graph crashes, restart from the last checkpoint
- **Human-in-the-loop** — pause the graph, let a human review/edit the state, then resume
- **Multi-session memory** — a user can continue a conversation days later

```
from langgraph.checkpoint.sqlite import SqliteSaver

# Add checkpointing
checkpointer = SqliteSaver.from_conn_string(':memory:')
graph = graph.compile(checkpointer=checkpointer)
```

Without checkpointing: if your 10-minute pipeline crashes at step 9, you restart from step 1. With checkpointing: restart from step 9.

Key Points to Remember:

- ★ Checkpointing = save state after each node
- ★ Enables: resume on failure, human-in-the-loop
- ★ Backends: SQLite, PostgreSQL, Redis
- ★ Critical for long-running production workflows

Answer:

Human-in-the-loop is a pattern where the AI workflow **pauses and waits for human approval or input** before continuing. LangGraph supports this with `interrupt_before` and `interrupt_after`.

Use cases: approve a purchase before executing, review a generated email before sending, validate AI analysis before publishing.

```
# Interrupt before quality gate runs
graph = graph.compile(
    checkpoint=checkpoint,
    interrupt_before=['quality_gate']
)

# Graph pauses here - human reviews
state = graph.get_state(thread_id)
# Human edits state...
graph.update_state(thread_id, edited_state)
# Resume from where it paused
graph.invoke(None, thread_id)
```

Human-in-the-loop = `interrupt_before/after` + checkpointing. The graph pauses, human reviews, then continues.

Key Points to Remember:

- ★ Human-in-loop = pause graph for human review
- ★ Needs checkpointing enabled
- ★ `interrupt_before='node_name'` pauses before that node
- ★ Use for: approve actions, validate outputs, edit state

Q41 How does LangGraph handle parallel execution?

Answer:

LangGraph supports parallel node execution using a **fan-out/fan-in pattern**. You add edges from one node to multiple nodes — they execute in parallel. A final node waits for all of them to complete.

```
# Fan-out: planner sends to all 3 workers
graph.add_edge('planner', 'search_agent')
graph.add_edge('planner', 'analysis_agent')
graph.add_edge('planner', 'examples_agent')

# Fan-in: all 3 feed into synthesizer
graph.add_edge('search_agent', 'synthesizer')
graph.add_edge('analysis_agent', 'synthesizer')
graph.add_edge('examples_agent', 'synthesizer')
```

In ResearchForge: the 3 worker agents run sequentially (in workers_node) for simplicity. True parallel execution would use this fan-out pattern.

Note: State merging requires careful design when multiple nodes write to the same field concurrently.

Key Points to Remember:

- ★ Fan-out = one node → multiple parallel nodes
- ★ Fan-in = multiple nodes → one merge node
- ★ LangGraph handles synchronisation automatically
- ★ Use `Annotated[list, operator.add]` for parallel list merging

Q42 How do you debug a LangGraph application?

Answer:

Three main debugging approaches:

- **Print statements** — add `print()` in each node to see what it receives and returns. Simple but effective.
- **LangSmith tracing** — set `LANGCHAIN_TRACING_V2=true`. Every node input/output, token count, and latency is captured and visible in the LangSmith dashboard.
- **`graph.stream()`** — iterate through stream output to see state after each node and identify where the problem starts.

```
# Debug with stream
for update in graph.stream(initial_state):
    node = list(update.keys())[0]
    state = update[node]
    print(f'After {node}:', state.get('status_log'))
```

LangSmith is the most powerful debugging tool — it shows the exact prompt sent to the LLM at each step.

Key Points to Remember:

- ★ Debug with: print statements, LangSmith, `stream()`
- ★ LangSmith shows exact prompts and LLM responses
- ★ `stream()` reveals which node produces bad output
- ★ Always check state after each node during development

Section 5 — Agentic AI & Multi-Agent Systems

Q43 What is Agentic AI? How is it different from a chatbot?

Answer:

Agentic AI refers to AI systems that can **autonomously plan, decide, and execute multi-step tasks** to achieve a goal — without a human directing each step.

Chatbot: responds to one message at a time. Stateless. Human drives every step.

Agentic AI: given a high-level goal, it breaks it down into sub-tasks, decides which tools to use, executes them in sequence, handles errors, and delivers a result.

Chatbot = reactive (answers questions). Agent = proactive (achieves goals).

ResearchForge is agentic: you give it a topic, it plans, searches, analyses, writes, and delivers a report — all autonomously.

Key Points to Remember:

- ★ Agent = plan + execute multi-step tasks autonomously
- ★ Chatbot = reactive, one turn at a time
- ★ Agent has: goal, tools, memory, decision-making
- ★ Key property: agent decides its own next step

Q44 What is the ReAct pattern? How is it used in agents?

Answer:

ReAct stands for **Reason + Act**. It is a prompting and agent architecture pattern where the agent alternates between thinking and doing:

- **Reason** — the agent thinks about what to do next (internal monologue)
- **Act** — the agent calls a tool or takes an action
- **Observe** — the agent reads the result of the action
- **Repeat** — reason about the observation, decide next action

Example trace in ResearchForge:

- Reason: 'I need to find web articles about AI in healthcare'
- Act: call search_web('AI healthcare 2024')
- Observe: '5 articles returned'
- Reason: 'Now I need to analyse these articles'
- Act: call analysis_agent...

ReAct = the fundamental loop of any AI agent. Every LangGraph node follows this implicitly.

Key Points to Remember:

- ★ ReAct = Reason → Act → Observe loop
- ★ Agent thinks before each action
- ★ Observe outcome, reason about next step
- ★ Foundation of most modern agent architectures

Q45

What is the Supervisor pattern in multi-agent systems?

Answer:

The Supervisor pattern has a **central orchestrator agent that directs specialised worker agents**. The supervisor receives the high-level goal, breaks it down, assigns sub-tasks to workers, collects results, and synthesises the final output.

In ResearchForge:

- Supervisor = Planner node — creates the TODO list and assigns agents
- Workers = Search, Analysis, Examples agents — execute specific tasks
- Synthesizer = collects all worker outputs

Supervisor pattern = divide and conquer with AI agents. Each agent is an expert at one thing.

Benefits: specialisation, context isolation, parallel execution, easier debugging (one agent at a time).

Key Points to Remember:

- ★ Supervisor = central agent that assigns tasks to workers
- ★ Workers = specialised agents for specific sub-tasks
- ★ Each agent has own context and focus
- ★ ResearchForge: Planner = supervisor, 3 agents = workers

Q46

What is Context Isolation in multi-agent systems?

Answer:

Context isolation means each sub-agent operates with only the information relevant to its specific task, rather than having access to the entire conversation history and all intermediate results.

Why it matters:

- Reduces LLM confusion — agent only sees its relevant task
- Reduces token usage — smaller context = cheaper
- Reduces hallucination — less irrelevant context = less noise

In ResearchForge: the analysis_agent only reads search_results.txt. It does not see the examples task or other agents' work. Its context is isolated to its job.

Context isolation = give each agent only what it needs to know. Nothing more.

Key Points to Remember:

- ★ Each agent has focused, minimal context
- ★ Reduces token cost and hallucination
- ★ Virtual File System enables selective context reading
- ★ Good agent design: single responsibility per agent

Q47

What is Context Offloading? Why is it needed?

Answer:

Context offloading is the technique of **storing intermediate results outside the LLM's context window** (in files, databases, or memory) so agents can process large amounts of information without hitting token limits.

Problem: if search_agent finds 10,000 words of search results and you try to pass all of it to every subsequent LLM call, you quickly exhaust the context window and costs explode.

Solution: search_agent saves results to virtual_files['search_results.txt']. Analysis_agent reads only what it needs. Synthesizer combines them selectively.

Context offloading = store it externally, retrieve selectively. Like saving a file instead of memorising everything.

Key Points to Remember:

- ★ Context offloading = store data outside LLM context
- ★ Solves: context window limits and high token costs
- ★ ResearchForge: virtual_files dict in state
- ★ Production: use vector databases for semantic retrieval

Q48

What is LLM-as-Judge? How is it implemented?

Answer:

LLM-as-Judge is a technique where you use an LLM to automatically evaluate the quality of another LLM's output. Instead of human reviewers, the LLM reads the output and scores it.

Implementation steps:

- Define clear scoring criteria in the prompt (what makes a good output?)
- Send the output to the LLM with the scoring prompt
- Parse the numeric score from the response using regex
- Use the score to route: accept output, retry, or escalate to human

ResearchForge quality_gate_node: sends report to Gemini, asks for 0-10 score, uses score in should_retry() to decide if synthesizer should run again.

Note: LLM-as-Judge is used in production at OpenAI, Anthropic, and Google for automated evaluation of model outputs.

Key Points to Remember:

- ★ LLM scores another LLM's output automatically
- ★ Define clear criteria in scoring prompt
- ★ Use regex to extract numeric score
- ★ Enables self-correcting pipelines without human review

Q49**What is the Virtual File System pattern? How does it work in LangGraph?****Answer:**

The Virtual File System (VFS) is an architectural pattern where agents simulate a file system using a Python dictionary stored in the LangGraph State. Agents write results to 'files' (dict keys) and read from them later.

```
# In tools/file_system.py
def write_file(vfs: dict, filename: str, content: str) -> dict:
    vfs[filename] = content
    return vfs

def read_file(vfs: dict, filename: str) -> str:
    return vfs.get(filename, '')

def read_all_files(vfs: dict) -> str:
    return '\n\n'.join(
        f'=== {k} ===\n{v}' for k, v in vfs.items()
    )
```

VFS = Python dict in state. Keys = filenames. Values = content. Zero setup needed.

Benefits: no external storage needed, persists across nodes, agents can selectively read only relevant files.

Key Points to Remember:

- ★ VFS = dict in state simulating a file system
- ★ write_file / read_file / read_all_files functions
- ★ Persists across all nodes via LangGraph state
- ★ Enables context offloading without external storage

Q50**What are CrewAI and AutoGen? How do they compare to LangGraph?****Answer:**

All three are frameworks for building multi-agent AI systems:

- **LangGraph** — low-level, graph-based, maximum control. You define every node, edge, and state field. Best for production systems where you need fine-grained control.
- **CrewAI** — higher-level abstraction. Define agents as 'crew members' with roles, goals, and tools. Less code but less control. Good for quick prototyping.
- **AutoGen** (Microsoft) — conversation-based multi-agent. Agents communicate by sending messages to each other like a group chat. Different mental model.

LangGraph = most production-ready. CrewAI = fastest to prototype. AutoGen = conversation-centric.

For interviews: mention that you chose LangGraph for ResearchForge because it provides direct control over state management and routing logic.

Key Points to Remember:

- ★ LangGraph = low-level, graph-based, maximum control
- ★ CrewAI = high-level, role-based agents
- ★ AutoGen = conversation/message passing model
- ★ LangGraph best for production; CrewAI for rapid prototyping

Q51 What is Tool Calling / Function Calling?

Answer:

Tool Calling (also called Function Calling) is an LLM feature where the model can request the execution of a specific function instead of generating text. The LLM sees a list of available tools with their descriptions, decides to call one, returns a structured call request, and the calling code executes the function and passes the result back.

Flow: LLM sees tool descriptions → decides to call tool X with parameters → returns JSON like {'tool': 'search_web', 'args': {'query': 'AI healthcare'}} → your code calls the function → returns result → LLM continues.

Tool calling is what separates a basic chatbot from a true AI agent — the ability to take actions in the real world.

Key Points to Remember:

- ★ LLM requests function execution instead of generating text
- ★ Model returns structured JSON with tool name + args
- ★ Your code executes the tool, passes result back
- ★ Foundation of all agentic AI systems

Q52 What is the difference between Synchronous and Asynchronous agents?

Answer:

Synchronous agents: each step waits for the previous one to complete before starting. Simple to implement, easy to debug. Most LangGraph examples are synchronous.

Asynchronous agents: multiple steps can run concurrently using Python's asyncio. Worker agents run in parallel, reducing total execution time.

```
# Async LangGraph node
async def search_node(state):
    result = await search_tool.ainvoke(query)
    return {'results': result}

# Async invoke
final = await graph.ainvoke(initial_state)
```

In ResearchForge: workers run sequentially (sync) for simplicity. For production: use async to run all 3 workers in parallel — 3x faster.

Key Points to Remember:

- ★ Sync = one step at a time, simple to debug
- ★ Async = concurrent execution, faster for parallel tasks
- ★ asyncio + ainvoke for async LangGraph
- ★ ResearchForge workers can be parallelised with async

Section 6 — ResearchForge Project — Deep Dive Questions

Q53

Walk me through your ResearchForge project from start to finish.

Answer:

ResearchForge is an autonomous multi-agent research system. The user enters a topic in a Streamlit UI. Here is the exact flow:

- **1. Planner Node** (planner.py): Gemini LLM receives the query and returns 3 tasks in structured format. `parse_todos()` converts them to a list of dicts saved in LangGraph state.
- **2. Workers Node** (workers.py): Three agents run sequentially. Search agent calls Tavily API and saves results to `virtual_files['search_results.txt']`. Analysis agent reads that file, sends to Gemini for analysis, saves to `analysis.txt`. Examples agent finds 3 real company cases, saves to `examples.txt`.
- **3. Synthesizer Node** (synthesizer.py): `read_all_files()` combines all 3 files. Gemini writes a 7-section report saved as `final_report` in state.
- **4. Quality Gate** (quality_gate.py): Gemini scores the report 0-10. If score < 5 and retries < 2, routes back to synthesizer. Otherwise routes to END.
- **5. UI** (app.py): Streamlit displays agent log, TODO checklist, file panels, quality badge, report, and download buttons for Markdown and PDF.

Key design decisions: LangGraph for stateful routing, Virtual File System for context management, LLM-as-Judge for quality control.

Key Points to Remember:

- ★ Know each node's input, process, and output
- ★ State flows through every node unchanged except updates
- ★ VFS solves context window limits
- ★ Quality gate = self-correcting pipeline

Q54

Why did you use LangGraph instead of just LangChain chains?

Answer:

Three specific reasons:

- **Retry loop:** The quality gate needs to route back to the synthesizer if score is low. This is a loop — impossible in linear LangChain chains.
- **Shared state:** All 4 agents need to read and write to the same data (todos, virtual_files, status_log). LangGraph State provides this cleanly as a single TypedDict passed through every node.
- **Conditional routing:** `should_retry()` returns 'retry' or 'done' based on quality score. LangGraph's conditional edges map these strings to nodes. A plain LangChain chain has no equivalent.

Simple rule: if your pipeline has any loops OR multiple agents sharing state → use LangGraph, not LangChain chains.

Key Points to Remember:

- ★ Reason 1: loop needed for retry logic
- ★ Reason 2: shared state across all agents
- ★ Reason 3: conditional routing based on score
- ★ LangChain chain = no loops, no shared state

Q55**Explain the parse_todos() function. Why is it important?****Answer:**

parse_todos() converts the LLM's raw text response into a structured Python list of dictionaries. This is a critical step because:

The LLM returns text like: 'TASK 1 | search_agent | Search for AI healthcare 2024'

parse_todos() splits each line by '|', extracts position [1] (agent name) and [2] (task description), and builds a dict with id, task, assigned_to, and status fields.

```
def parse_todos(llm_response: str) -> list:
    todos = []
    for line in llm_response.strip().split('\n'):
        if '|' not in line: continue
        parts = [p.strip() for p in line.split('|')]
        if len(parts) >= 3:
            todos.append({
                'id': len(todos)+1,
                'task': parts[2],
                'assigned_to': parts[1],
                'status': 'pending'
            })
    return todos
```

parse_todos() is a custom Output Parser. It converts unstructured LLM text to structured Python data that workers can use.

Key Points to Remember:

- ★ Custom output parser for structured LLM output
- ★ Splits by | to extract agent name and task
- ★ Always validate: check len(parts) >= 3
- ★ Fallback: hardcoded todos if parsing fails

Q56

How does the Virtual File System work in your project?

Answer:

The Virtual File System is a plain Python dictionary stored in the LangGraph state under the key 'virtual_files'. It uses filenames as keys and file content as values.

Four functions in tools/file_system.py:

- write_file(vfs, 'search_results.txt', content) → saves to dict
- read_file(vfs, 'search_results.txt') → returns content or empty string
- list_files(vfs) → returns list of all filenames
- read_all_files(vfs) → combines all files with === filename === headers

Flow: search_agent saves search_results.txt → analysis_agent reads it → examples_agent saves examples.txt → synthesizer reads all three with read_all_files().

The VFS solves context window limitations — agents store large outputs externally and only retrieve what they need, avoiding token limit issues.

Key Points to Remember:

- ★ VFS = Python dict in state (zero external setup)
- ★ write_file / read_file / read_all_files
- ★ Persists across all 4 nodes via state
- ★ Solves context window limits between agents

Q57

How does the Quality Gate work? Why did you include it?

Answer:

The quality gate node uses LLM-as-Judge: it sends the first 2000 characters of the final report to Gemini with a scoring prompt asking for a 0-10 number. Regex extracts the score. should_retry() checks if score < 5 and retries < 2 — if so, routes back to synthesizer.

Why included: without quality control, the pipeline might produce a vague or poorly structured report and present it as the final answer. The quality gate makes the system self-correcting.

Why temperature=0 for quality gate: scoring needs consistency, not creativity. Same report should always get the same score.

Quality Gate = self-correcting mechanism. Makes the AI system responsible for its own output quality.

Note: Score threshold set to 5 (not 7) to account for free-tier model limitations on the first attempt.

Key Points to Remember:

- ★ LLM scores report 0-10 with specific criteria
- ★ should_retry() = router function for conditional edge
- ★ Max 2 retries to prevent infinite loops
- ★ temperature=0 for consistent scoring

Q58**What challenges did you face and how did you solve them?****Answer:**

Three main technical challenges:

- **Challenge 1 — Model not found error:** gemini-1.5-flash was deprecated. Fixed by upgrading to gemini-2.5-flash using VS Code's Find & Replace across all 4 files.
- **Challenge 2 — Rate limit errors (429):** Gemini's free tier allows only 20 requests/day. Solution: integrated Groq (14,400 req/day free) and Ollama (local, unlimited) as fallback providers. Only 4 files need changing.
- **Challenge 3 — PDF generation crash:** fpdf2 v2.8 leaves cursor at x=200 (page edge) after each multi_cell() call. The next call finds no horizontal space and crashes. Fixed with a _mcell() helper function that resets pdf.set_x(lm) before every text write.

Key interview insight: showing you debugged real technical problems demonstrates engineering maturity, not just using a framework.

Key Points to Remember:

- ★ Model deprecation → update model name in 4 files
- ★ Rate limits → Groq (free, 14K/day) or Ollama (local)
- ★ PDF crash → cursor bug in fpdf2, fixed with set_x(lm)
- ★ Always mention: problem → root cause → solution

Q59**How would you improve ResearchForge for production?****Answer:**

Six concrete improvements for production deployment:

- **True parallel workers:** use async/await so all 3 worker agents run simultaneously. Current implementation is sequential.
- **LangGraph checkpointing:** add PostgreSQL checkpointer so long research tasks survive crashes and can be resumed.
- **Vector database:** replace the simple VFS with ChromaDB for semantic search across stored research results.
- **Source citations:** add a citation agent that extracts and formats references from search results.
- **User accounts:** save research history per user using LangGraph's thread_id system.
- **Streaming UI:** use graph.stream() to update Streamlit progressively as each agent completes.

In interviews: production improvements show you understand real engineering, not just demo-level code.

Key Points to Remember:

- ★ Async workers = parallel execution = 3x faster
- ★ Checkpointing = resume on failure
- ★ Vector DB = semantic retrieval for large data
- ★ Streaming UI = better user experience

Answer:

state.py defines ResearchState, a TypedDict — a Python type that creates a dictionary with declared, type-checked fields. It is the schema for the shared memory that all LangGraph nodes read and write.

TypedDict benefits: IDE autocomplete, type checking with mypy, self-documenting code, LangGraph uses it to validate state updates.

Fields in ResearchState:

- user_query: str — the original research topic
- todos: List[TodoItem] — 3 tasks created by planner
- virtual_files: dict — files saved by worker agents
- final_report: Optional[str] — the complete report
- quality_score: Optional[float] — score from quality gate
- retry_count: int — tracks synthesis attempts
- status_log: List[str] — activity log shown in UI

Every field that any agent reads or writes must be declared in state.py. It is the single source of truth for the pipeline's memory.

Key Points to Remember:

- ★ TypedDict = dict with declared types = schema
- ★ All agent-shared data must be in state
- ★ Optional[str] = field may not exist yet
- ★ status_log = bridge between backend agents and UI

Answer:

The connection is one line: `final_state = graph.invoke(initial_state)`. The UI builds the initial state dict from the user's query, calls `invoke()`, waits for the graph to complete, then reads the returned final state to populate the display.

```
# In app.py – the single connection point
graph = build_graph() # imports compiled graph

initial_state = {
    'user_query': query.strip(),
    'todos': [], 'virtual_files': {},
    'final_report': None,
    'quality_score': None,
    'retry_count': 0, 'status_log': []
}

final_state = graph.invoke(initial_state)
```

After `invoke()` returns: `render_agent_status()` reads `status_log`, `render_virtual_files()` reads `virtual_files`, `render_quality_badge()` reads `quality_score`, `render_report()` reads `final_report`.

The UI only calls `invoke()` once. Everything else is reading fields from the returned state dictionary.

Key Points to Remember:

- ★ UI builds `initial_state` → calls `graph.invoke()`
- ★ Graph runs all nodes → returns final state
- ★ UI reads fields from final state to render sections
- ★ `components.py` renders each section from state fields

Q62

Why did you choose Streamlit for the UI? What are its limitations?

Answer:

Streamlit was chosen because: it runs as pure Python (no HTML/CSS/JS needed), it integrates naturally with Python data science and AI code, deployment to Streamlit Cloud is one click from GitHub, and it supports markdown rendering natively (perfect for displaying research reports).

Limitations of Streamlit:

- **Re-runs entire script on every interaction** — can be slow for complex apps
- **Not suitable for real-time streaming** without advanced patterns
- **Limited customisation** compared to React/Vue
- **Single-threaded** — cannot handle multiple concurrent users well at scale
- **CSS variables** like `var(--color-text-primary)` don't work reliably on Hugging Face Spaces — must use hardcoded colors

For an internship/portfolio project: Streamlit is ideal. For production SaaS: use FastAPI backend + React frontend.

Key Points to Remember:

- ★ Streamlit = pure Python UI, great for AI demos
- ★ Pros: fast to build, GitHub deploy, markdown support
- ★ Cons: re-renders on every interaction, limited real-time
- ★ Production: FastAPI + React for scale

Section 7 — Quick Revision Summary Table

Study this table the night before your interview. Each row = one concept you must be able to explain in 60 seconds.

Concept	One-line definition	Where in project	Interview hook
Generative AI	Creates new content by learning patterns	Full pipeline output	vs Predictive AI (classify vs create)
Foundation Model	Large pre-trained model, fine-tuned for tasks	Gemini 2.5 Flash used	Examples: GPT-4, Gemini, Claude, LLaMA
Transformer	Neural net architecture using self-attention	Inside Gemini	2017 'Attention Is All You Need' paper
Token	Basic unit of text LLMs process	~750 tokens per request	1 token ≈ 0.75 words, context window = max tokens
Temperature	Controls randomness of LLM output	0 for quality gate, 0.3 planned	0=deterministic, 1+=creative
Prompt Engineering	Design prompts for reliable structured outputs	Every node prompt	Structured format for parse_todos()
RAG	Retrieve docs + LLM generation	Tavily search is RAG-like	Solves hallucination + knowledge cutoff
Hallucination	LLM generates confident wrong info	Quality gate catches it	Cause: statistical prediction, not retrieval
RLHF	Train LLM to align with human preferences	Gemini is RLHF-trained	Makes models helpful, harmless, honest
LangChain	Framework for LLM apps (linear chains)	Underlying library	LCEL pipe operator
LangGraph	Graph-based stateful agent workflows	Core framework	Nodes + Edges + State + Conditional routing
StateGraph	LangGraph's main class using TypedDict	graph_builder.py	Shared whiteboard for all agents
Node	Function (state) → state	All 4 main files	Read state, do work, return updates
Edge	Connection between nodes	graph_builder.py	Regular: always. Conditional: based on function
Conditional Edge	Routes to different nodes based on output	Quality gate routing	should_retry() returns 'retry' or 'done'
Retry Loop	Conditional edge pointing back to earlier node	Synthesizer retry	Always add retry_count guard
compile()	Locks graph into runnable object	graph_builder.py	Must call before .invoke()
Checkpointing	Save state after each node to persistent store	state in current version	Enables resume on failure
ReAct Pattern	Reason → Act → Observe loop	workers.py agents	Foundation of all AI agents
Supervisor Pattern	Central agent directs specialised workers	Planner → Workers	Divide and conquer with agents
Context Offloading	Store intermediate results outside LLM context	Virtual File System	Solves context window limits
Virtual File System	Python dict in state simulating file storage	tools/file_system.py	write_file / read_file / read_all_files
LLM-as-Judge	LLM scores another LLM's output	quality_gate.py	Self-correcting pipelines
Tool Use	LLM requests execution of a function	search_tool.py	Gives agents ability to act in real world
Embedding	Numerical vector representing text meaning	Used in RAG/search	Similar meaning = similar vectors
Vector Database	Stores embeddings for similarity search	Mentioned as improvement	FAISS, Pinecone, ChromaDB
Fine-tuning	Extra training on task-specific dataset	Not used (prompting used)	Use when prompting insufficient
parse_todos()	Custom output parser for LLM response	planner.py	Splits by to get structured dicts
Groq	Fast free LLM API (14K req/day)	Fallback provider	Best free alternative to Gemini

Prepared by Vishal Lazrus | github.com/vishal815 | ResearchForge — Autonomous Multi-Agent Research System